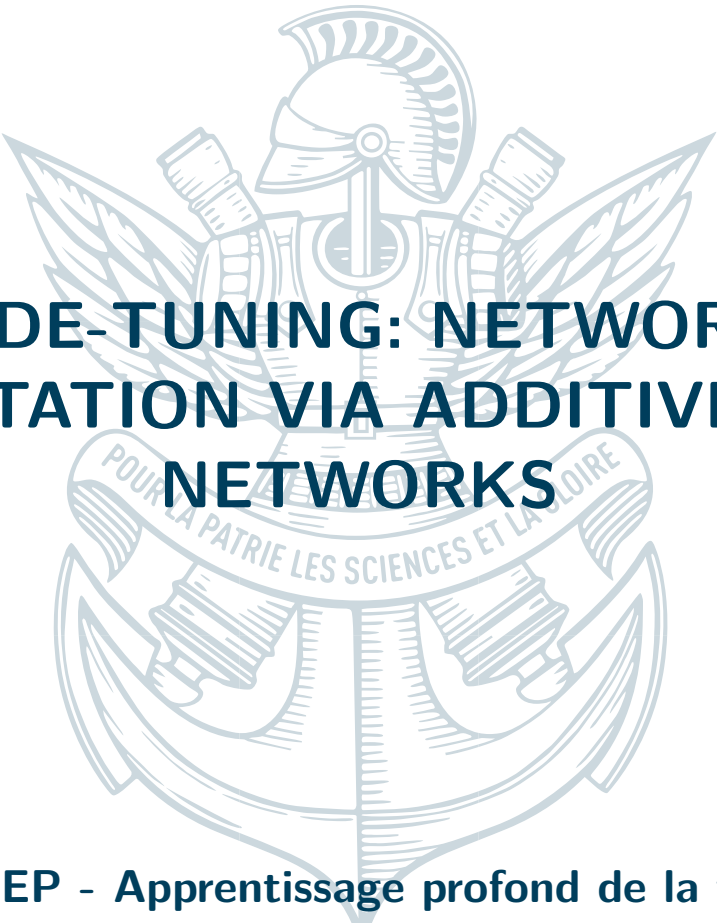




SIDE-TUNING: NETWORK ADAPTATION VIA ADDITIVE SIDE NETWORKS

**APM52183EP - Apprentissage profond de la théorie à la
pratique**

Projet 2025-2026

11 mars 2026

Emile DRIJARD-MAZZINI, Andréa BOURELLY, Benjamin WILHELM &
Nolan COULON

Promotion X2023

TABLE DES MATIÈRES

Introduction	3
1 Reproduction et comparaison	3
1.1 Reproduction des résultats	3
1.2 Comparaison avec les autres modèles	4
2 Influence des méthodes et paramètres	5
2.1 Optimiseur et learning rate	5
2.2 Merge method	6
2.3 Structure du réseau	7
3 Extension à PatchCamelyon : shift de domaine sévère	8
3.1 Acte 1 — Régime de faibles données et overfitting	8
3.2 Acte 2 — Ablation : fusion, taille et base	9
3.3 Acte 3 — Benchmark PEFT et résultats finaux	9
Conclusion	10
Références	11
Annexes	12

INTRODUCTION

L'adaptation de réseaux de neurones pré-entraînés est une problématique centrale en apprentissage automatique, notamment dans des contextes où les données sont limitées et où l'apprentissage doit être continu. Les approches classiques telles que le *fine-tuning* ou l'extraction de caractéristiques fixes souffrent respectivement de **l'oubli catastrophique** et de **performances sous-optimales**.

Le projet initial [6] propose donc l'alternative du **side-tuning**, une méthode qui adjoint un réseau léger et entraînable à un réseau pré-entraîné figé, dont les sorties sont fusionnées selon une méthode choisie. Cette approche a pour objectif d'éviter l'oubli catastrophique et de réduire le surapprentissage tout en restant simple à mettre en œuvre.

Dans ce projet, nous reproduisons et étendons les expériences du papier sur le benchmark iCIFAR — CIFAR-100 découpé en 10 tâches séquentielles de 10 classes chacune — avec un ResNet-44 pré-entraîné sur CIFAR-10 comme base et un FCN à 4 couches distillé depuis ce ResNet-44 comme side network. Nous commençons par **reproduire et comparer** les résultats de l'article [6] aux baselines de référence (fine-tuning, extraction de features, réseaux indépendants). Nous analysons ensuite **l'influence des choix méthodologiques** - optimiseur, méthode de fusion, structure du réseau - pour mieux comprendre ce qui gouverne les performances du modèle. Enfin, nous **étendons la méthode** à deux nouveaux datasets (STL-10 et FashionMNIST) pour évaluer sa capacité de généralisation hors de son contexte d'origine.

1 REPRODUCTION ET COMPARAISON

1.1 REPRODUCTION DES RÉSULTATS

- DU SCRIPT DÉMO AU PIPELINE COMPLET

Le dépôt officiel fournit un script `demo_icifar.py` conçu pour une prise en main rapide : il instancie directement le modèle `LifelongSidetuneNetwork`, charge les données iCIFAR et exécute une boucle d'entraînement minimaliste en une centaine de lignes. Ce script constitue une bonne introduction mais ne reflète pas le pipeline expérimental complet du papier.

Le pipeline utilisé dans ce projet repose sur **Sacred**, un framework de gestion d'expériences qui introduit plusieurs différences importantes. Les hyperparamètres sont centralisés dans des fichiers de configuration (`configs/`), ce qui garantit la reproductibilité exacte de chaque run et permet de les modifier depuis la ligne de commande sans toucher au code. Le suivi des métriques est plus complet : en plus de l'accuracy finale, le pipeline enregistre l'historique d'entraînement tâche par tâche, l'oubli par tâche (*forgetting*), et les écarts entre meilleure accuracy et accuracy finale. Enfin, les résultats sont sérialisés dans des fichiers `result_log.pkl` permettant une analyse post-hoc détaillée.

C'est ce pipeline complet qui est utilisé pour toutes les expériences de ce rapport, garantissant une comparaison rigoureuse entre méthodes dans des conditions strictement identiques.

- RÉSULTATS OBTENUS

Nous reproduisons Side-Tuning sur le benchmark iCIFAR dans les conditions du papier original : ResNet-44 pré-entraîné sur CIFAR-10 comme base, FCN-4 distillé comme side network, 10 tâches séquentielles de 10 classes chacune issues de CIFAR-100. Nous obtenons une accuracy moyenne finale de **67.3%**, sans aucun oubli catastrophique, ce qui reproduit fidèlement la tendance centrale du papier.

L'écart avec l'accuracy annoncée dans le papier ($\sim 76\%$) s'explique par une différence de protocole : le script `demo_icifar.py` effectue plusieurs passes sur chaque tâche lors de l'entraînement, tandis que notre pipeline Sacred utilise strictement **une epoch par tâche** conformément au protocole lifelong learning, où chaque tâche n'est vue qu'une seule fois. Cette différence illustre précisément l'apport du pipeline complet : une traçabilité exacte des conditions expérimentales qui permet d'identifier et d'expliquer de tels écarts.

1.2 COMPARAISON AVEC LES AUTRES MODÈLES

Le graphique 1 résume les performances finales des quatre méthodes sur iCIFAR. Les résultats montrent que les méthodes qui gèlent la base (features, sidetune, independent) surpassent massivement le fine-tuning, dont la moyenne finale de **22.8%** montre l'oubli catastrophique.

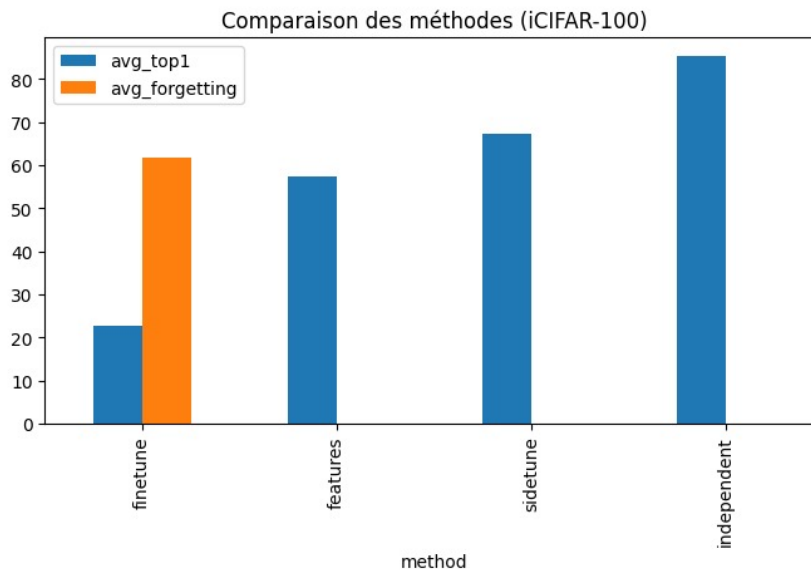


FIGURE 1 – Accuracy moyenne finale et oubli moyen par méthode sur iCIFAR-100.

Fine-tuning. Avec une accuracy moyenne finale de **22.8%** et un oubli moyen de **61.8%**, le fine-tuning illustre parfaitement le problème de l'oubli catastrophique. Le graphique *Best vs Final* (voir annexe) révèle que chaque tâche atteint une bonne performance lors de son apprentissage (entre 72% et 88%), avant d'être presque entièrement oubliée dès que les tâches suivantes sont introduites. Seule T9, dernière tâche apprise, conserve sa performance (88%). L'oubli est particulièrement prononcé pour les premières tâches (T0 : 79.5 %, T3 : 79.0%), qui ont subi le plus grand nombre de mises à jour destructrices.

Feature Extraction. En gelant entièrement la base et en n'entraînant qu'un classifieur linéaire, cette méthode élimine complètement l'oubli catastrophique — la performance de chaque tâche reste stable une fois apprise (voir annexe). La moyenne finale atteint **57.3%**, soit plus du double du fine-tuning. Cependant, la capacité d'adaptation est limitée : les features du ResNet-44, optimisées pour CIFAR-10, ne sont combinées que linéairement, ce qui plafonne les performances sur les nouvelles classes de CIFAR-100.

Side-Tuning. Side-Tuning atteint une accuracy moyenne finale de **67.3%**, soit un gain de **+10 points** par rapport à l'extraction de features. Comme l'illustre le graphique *Best vs Final* (voir annexe), les performances finales et maximales coïncident parfaitement pour chaque tâche — preuve que le gel de la base garantit une

absence totale d'oubli. Ce gain sur les features s'explique par la capacité non-linéaire du side network, qui apprend des représentations complémentaires adaptées à chaque tâche au-delà de ce qu'une simple projection linéaire peut capturer.

Réseaux indépendants. Avec **85.4%** de moyenne finale, les réseaux indépendants constituent la borne supérieure de cette comparaison. Ils ne souffrent d'aucun oubli par construction — chaque tâche dispose de son propre réseau complet — et atteignent les meilleures performances sur toutes les tâches (entre 82% et 90%). Cependant, ce résultat a un coût : le nombre de paramètres croît linéairement avec le nombre de tâches, rendant cette approche bien plus lourde.

Bilan. Le tableau 1 synthétise les résultats. Side-Tuning offre le meilleur compromis : il surpasse de loin le fine-tuning et l'extraction de features, ne souffre d'aucun oubli catastrophique, et n'est qu'à **18 points** des réseaux indépendants tout en utilisant une fraction de leurs paramètres.

Méthode	Acc. moy. finale	Oubli moyen	Oubli catastrophique	Paramètres
Independent	85.4%	0%	Non	$\times 10$
Side-Tuning	67.3%	0%	Non	$\times 1.3$
Features	57.3%	0%	Non	$\times 1.001$
Fine-tuning	22.8%	61.8%	Oui	$\times 1$

TABLE 1 – Comparaison des quatre méthodes sur iCIFAR-100.

2

INFLUENCE DES MÉTHODES ET PARAMÈTRES

2.1 OPTIMISEUR ET LEARNING RATE

Nous avons comparé trois optimiseurs (SGD, Adam, AdamW) sur une gamme de learning rates allant de 10^{-4} à 10^{-1} , en faisant varier également le weight decay. Le critère principal est l'accuracy moyenne sur toutes les tâches (*avg_top1*), le forgetting étant nul dans tous les cas (tableau 2.1).

Optimiseur	lr	Weight decay	Momentum	Avg top-1 (%)
Adam	10^{-2}	0	—	71.63
SGD	10^{-1}	10^{-4}	0.9	71.52
SGD	10^{-2}	10^{-4}	0.9	69.01
AdamW	10^{-3}	10^{-2}	—	67.44
Adam	10^{-3}	0	—	67.44
Adam	10^{-3}	10^{-4}	—	67.42
Adam	10^{-3}	10^{-5}	—	67.32
AdamW	10^{-3}	10^{-3}	—	67.27
SGD	10^{-3}	10^{-4}	0.9	53.73
AdamW	10^{-4}	10^{-2}	—	39.97
Adam	10^{-4}	0	—	39.96

TABLE 2 – Comparaison des optimiseurs et learning rates sur iCIFAR

Adam. Adam s'avère l'optimiseur le plus performant dans cette expérience. Avec $lr = 10^{-2}$, il atteint la meilleure performance avec **71.63%**. À $lr = 10^{-3}$, il se stabilise autour de 67.3–67.4% quelle que soit la valeur de weight decay testée ($0, 10^{-5}, 10^{-4}$), ce qui témoigne d'une bonne insensibilité à ce hyperparamètre secondaire.

SGD avec momentum. Le SGD avec momentum se révèle compétitif à condition d'utiliser un learning rate suffisamment élevé. La configuration $lr = 10^{-1}$, momentum = 0.9 atteint **71.52%**, très proche du meilleur Adam, suivie de $lr = 10^{-2}$ à 69.01%. En revanche, le SGD s'avère très sensible à ce paramètre : descendre à $lr = 10^{-3}$ fait chuter l'accuracy à seulement 53.73%, soit une dégradation de près de 18 points.

AdamW. AdamW produit des résultats légèrement inférieurs à Adam dans toutes les configurations testées (67.27–67.44% à $lr = 10^{-3}$), sans apporter de gain notable malgré la régularisation explicite par weight decay.

Effet d'un learning rate trop faible. Un learning rate de 10^{-4} pénalise fortement tous les optimiseurs : Adam et AdamW tombent autour de 39.96–39.97%, confirmant que le réseau *side* ne converge pas suffisamment en un nombre fixe d'époques.

Conclusion. Nous retenons **Adam** avec $lr = 10^{-2}$ et sans weight decay comme configuration optimale, offrant la meilleure accuracy (71.63%). SGD avec $lr = 10^{-1}$ et momentum = 0.9 constitue une bonne alternative (71.52%), avec une sensibilité plus forte au choix du learning rate.

2.2 MERGE METHOD

Nous testons huit opérateurs de fusion sur iCIFAR, en fixant l'optimiseur Adam avec $lr=0.01$. Les variantes explorées sont les suivantes : fusion scalaire (**Alpha**), modulateur (**FiLM**), multiplicative (**Product**), et non-linéaires de capacité variable (**MLP**, **MLP2**, **ResMLP2**, **MLP hidden a/b**).

Les résultats montrent que les fusions **non-linéaires de complexité modérée** — MLP et FiLM — surpassent les autres variantes, comme le montre la figure 2. La fusion Alpha, bien que simple et interprétable (les alphas appris convergent en moyenne autour de 0.85, indiquant que le modèle s'appuie majoritairement sur la base figée), se révèle insuffisamment expressive pour capturer les interactions entre les représentations de la base et du side network. À l'inverse, les architectures les plus complexes (MLP2, ResMLP2, MLP hidden) ne traduisent pas leur capacité supplémentaire en gain de performance, probablement en raison d'un surapprentissage sur le faible nombre de paramètres disponibles par tâche.

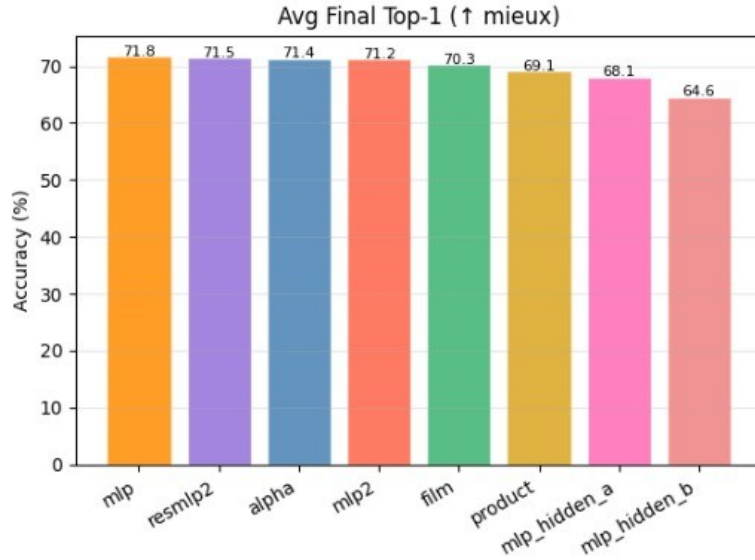


FIGURE 2 – Comparaison des méthodes de fusion sur iCIFAR — accuracy moyenne finale par opérateur.

La fusion **MLP** est retenue comme configuration optimale pour la suite des expériences. Ce résultat suggère que la fusion dans Side-Tuning bénéficie d’une non-linéarité, mais que l’essentiel de la capacité d’adaptation doit rester dans le side network lui-même plutôt que dans l’opérateur de fusion.

2.3 STRUCTURE DU RÉSEAU

En combinant les meilleurs choix des axes précédents (Adam lr = 10^{-2} , **merge_mlp**), nous avons étudié l’influence de l’architecture du side network sur 7 configurations (tableau 3).

Expérience	Description	Val loss	Catégorie
side_finetune	ResNet-44 fine-tuné complet	0.490	Fine-tune
side_fcn4_pretrained	FCN4 pré-entraîné (distillé)	0.558	FCN4
side_fcn4_scratch	FCN4 entraîné from scratch	0.635	FCN4
side_independent	Réseau indépendant par tâche	0.651	Référence
side_resnet44	ResNet-44 comme side network	0.682	ResNet
side_none	Pas de side (features seules)	0.802	Ablation
side_linear	Side linéaire uniquement	0.802	Ablation

TABLE 3 – Comparaison des architectures de side network (Adam lr= 10^{-2} , merge_mlp).

Meilleure configuration side-tuning. Le side network **side_resnet44** — un second ResNet-44 utilisé comme side network — atteint la meilleure performance parmi les approches sans oubli, avec **75.14%** d’accuracy moyenne et un oubli moyen de seulement **1.75%**. Ce léger oubli (contre 0% avec le FCN4) s’explique par la plus grande capacité du réseau, qui peut légèrement interférer entre tâches lors de l’entraînement séquentiel.

Rôle du pré-entraînement. La comparaison entre **side_fcn4_pretrained** et **side_fcn4_scratch** révèle l’importance de l’initialisation : le FCN4 pré-entraîné (distillé depuis le ResNet-44) obtient une val loss de 0.558

contre 0.635 pour son homologue entraîné from scratch, soit un gain significatif. Ceci confirme l'intérêt de partir d'une initialisation informée plutôt qu'aléatoire.

Ablations. Les configurations `side_none` et `side_linear` — respectivement sans side network et avec un simple side linéaire — obtiennent la même val loss (0.802), ce qui confirme qu'un side network non-linéaire est indispensable au gain de performance observé.

Bilan. La configuration optimale retenue est `side_resnet44` avec Adam ($lr = 10^{-2}$) et `merge_mlp`, atteignant 75.14% d'accuracy moyenne sur iCIFAR-100 — un gain de +7.8 points sur la configuration de base du papier (67.3%).

3

EXTENSION À PATCHCAMELYON : SHIFT DE DOMAINE SÉVÈRE

Les expériences précédentes (§1-2) se situent dans un contexte favorable : iCIFAR-100 est construit sur des images naturelles proches de la distribution d'ImageNet. Pour évaluer la robustesse de side-tuning dans des conditions moins favorables, nous l'appliquons à PCAM [7], un dataset de classification binaire d'images histologiques (96×96 px, hématoxyline-éosine) de ganglions lymphatiques (262 144 images train, parfaitement équilibré 50/50, voir annexe 12).

Pourquoi ce dataset constitue un défi. Les features apprises sur ImageNet — textures d'objets, contours nets, structures scéniques — sont peu pertinentes pour distinguer des morphologies cellulaires à l'échelle microscopique. De plus, la tâche est invariante à l'orientation (flips H/V), et les structures discriminantes sont locales et à fine échelle, partiellement perdues dans les couches profondes d'un ResNet. C'est précisément le scénario où side-tuning est censé exceller : une base peu adaptée au domaine cible.

3.1 ACTE 1 — RÉGIME DE FAIBLES DONNÉES ET OVERFITTING

Nous comparons features, fine-tuning et side-tuning sur 10k images avec ResNet-18, en faisant varier le nombre d'epochs (voir annexe 13).

Méthode	5 ep (val)	20 ep (val)	Train (20 ep)	Gap	Params
Features	79.3%	76.7%	77.1%	0.4 pt	1 026
Side-Tuning	81.2%	81.2%	84.3%	3.1 pt	160k
Fine-Tuning	87.5%	79.2%	92.9%	13.7 pt	11.2M

TABLE 4 – Baselines sur PCAM (10k train, 2k val, ResNet-18). Gap = écart train/val.

À 5 epochs le fine-tuning domine grâce à ses 11.2M paramètres. Mais à 20 epochs il s'effondre à 79.2% (gap train/val de 13.7 pts), signe d'un overfitting sévère que le format séquentiel d'iCIFAR masquait. Side-tuning, contraint à 160k paramètres, reste stable à 81.2% quelle que soit la durée d'entraînement. L'alpha appris décroît de 0.54 à 0.38 entre 5 et 20 epochs : le modèle apprend progressivement à se fier davantage au side network (adapté au domaine) qu'à la base ImageNet.

3.2 ACTE 2 — ABLATION : FUSION, TAILLE ET BASE

Sur 20k images, 10 epochs, nous explorons trois axes de conception indépendamment.

Méthode de fusion (Base=RN18, Side=Medium). La fusion MLP — concaténation des features base et side, projection non-linéaire $\mathbb{R}^{1024} \rightarrow \mathbb{R}^{512}$ — surpasse la fusion scalaire α de **+6.65 pts** (83.7% vs 77.1%). Ce résultat fait écho à la § 2.2 sur iCIFAR : sur les deux datasets, la non-linéarité de la fusion est plus déterminante que la capacité brute du réseau latéral. La fusion Product (élément par élément) offre un intermédiaire à 81.3%.

Taille du side network (Base=RN18, Merge=Alpha). Side Large (4 couches, 256 filtres, 1.1M params) atteint 81.6% mais souffre d’un overfitting marqué à 20 epochs (79.0%, gap de 10.9 pts). Side Medium présente la meilleure robustesse : variation de moins d’un point entre 5 et 20 epochs.

Réseau de base (Side=Medium, Merge=Alpha). ResNet-50 apporte +1.4 pts sur ResNet-18 (82.5% vs 81.2%). L’alpha converge plus bas (0.26 vs 0.36), confirmant que le modèle compense automatiquement une base moins pertinente en s’appuyant davantage sur le side network : ce mécanisme d’auto-adaptation, absent du fine-tuning, est une propriété structurelle du side-tuning.

La combinaison optimale — ResNet-50 + MLP + side Large, 10 epochs — est ensuite validée sur une grille 3×3 (side size $\times$ epochs, annexe 14). La meilleure configuration atteint **85.9%** à 20k images. Le side Large nécessite précisément 10 epochs : à 20 epochs, il chute à 79.0% (train = 89.9%), illustrant l’existence d’un *sweet spot* entre sous-apprentissage et overfitting.

3.3 ACTE 3 — BENCHMARK PEFT ET RÉSULTATS FINAUX

Fort des enseignements de l’ablation, nous confrontons side-tuning à six méthodes PEFT dans des conditions strictement identiques (Base=RN50, 20k images, 10 epochs).

Méthode	Val Acc	Train Acc	Val Loss	Params	Acc/M
Adapter	86.7%	92.6%	0.0064	507k	170.8
Fine-Tuning	86.1%	95.9%	0.0119	23.5M	3.7
Side-Tuning	82.7%	88.4%	0.0071	2.9M	28.2
BitFit	82.7%	85.8%	0.0067	31k	2696
Prompt Tuning	81.7%	83.4%	0.0069	673k	121
LoRA	81.6%	81.9%	0.0071	4k	19 912
Features	80.9%	81.9%	0.0072	4k	19 729

TABLE 5 – Benchmark PEFT sur PCAM (Base=RN50, 20k train, 10 epochs). *Acc/M* = accuracy par million de paramètres.

Quatre enseignements se dégagent (voir aussi annexes 15–16) :

- **Adapter domine.** Avec 86.7% en validation, la meilleure val_loss (0.0064) et seulement 507k paramètres (2% de ResNet-50), les Adapter Layers constituent la solution optimale pour PCAM. Leurs bottlenecks résiduels insérés après chaque stage ResNet permettent une adaptation précise aux features histologiques à chaque niveau de représentation.
- **Fine-tuning souffre d’overfitting.** Malgré 86.1% en validation, le gap train/val de 9.75 pts et la val_loss de 0.0119 (la pire du benchmark) révèlent un overfitting persistant même à 20k images.
- **BitFit : la surprise.** En n’entraînant que les biais de ResNet-50 (0.13% des paramètres, soit 31k), BitFit égale side-tuning à 82.7%. Une simple recalibration des activations suffit donc pour une adaptation partielle de domaine.

- **LoRA : inadapté aux CNN.** Conçu pour les Transformers (matrices Q, K, V), LoRA ne trouve pas suffisamment de surface d'adaptation dans les rares couches **fc** des bottlenecks ResNet. Son score (81.6%) reste proche de la baseline Features.

Confirmation sur 50k images. Les 3 meilleures méthodes sont relancées sur 50k images : Adapter **88.4%**, Fine-Tuning 87.6%, Side-Tuning 85.6%. La hiérarchie se confirme. Adapter consolide sa première place avec un gap train/val maîtrisé de 5.4 pts contre 9.5 pour Fine-Tuning. Side-Tuning progresse de +2.95 pts grâce aux données supplémentaires, confirmant sa sensibilité positive au volume d'entraînement.

Bilan. Sur ce domaine éloigné d'ImageNet, side-tuning n'est pas la méthode la plus performante mais offre le meilleur **compromis stabilité / efficacité paramétrique / absence d'oubli**. Là où le fine-tuning s'effondre avec la durée d'entraînement, side-tuning converge de façon monotone. L'alpha décroît avec la quantité de données ($0.54 \rightarrow 0.33$), révélant un mécanisme d'auto-adaptation absent de toutes les autres méthodes testées.

CONCLUSION

Ce projet a permis de reproduire, analyser et étendre la méthode Side-Tuning dans trois contextes progressivement plus exigeants.

La **reproduction sur iCIFAR** confirme les résultats du papier : Side-Tuning élimine l'oubli catastrophique tout en surpassant largement le fine-tuning et l'extraction de features, avec un coût en paramètres bien inférieur aux réseaux indépendants. C'est le compromis central que le papier revendique, et nos expériences le valident.

L'**étude des hyperparamètres** révèle que le choix de l'optimiseur et de la méthode de fusion ont un impact significatif sur les performances — plus important que ce que le papier original laisse supposer. La configuration optimale identifiée (Adam $lr=10^{-2}$, fusion MLP, side ResNet-44) atteint 75.14%, soit un gain de +7.8 points sur la baseline de référence, ce qui suggère que Side-Tuning est sous-exploité dans sa formulation d'origine.

L'**extension à PatchCamelyon** met en lumière un avantage souvent négligé de la méthode : sa **robustesse à l'overfitting**. Sur des domaines éloignés de la distribution d'entraînement de la base, là où le fine-tuning se dégrade avec la durée d'entraînement, Side-Tuning converge de façon stable. Face aux méthodes PEFT modernes, il n'est pas systématiquement le plus performant, mais il offre le meilleur compromis entre stabilité, efficacité paramétrique et absence d'oubli — trois qualités particulièrement précieuses dans des scénarios de données limitées ou de domaines hétérogènes.

RÉFÉRENCES

- [1] Zhang, J., et al. (2019). *Side-Tuning : A Baseline for Network Adaptation via Additive Side Networks*. ECCV 2020.
- [2] Veeling, B. S., et al. (2018). *Rotation equivariant CNNs for digital pathology*. MICCAI 2018.
- [3] Hu, E. J., et al. (2022). *LoRA : Low-Rank Adaptation of Large Language Models*. ICLR 2022.
- [4] Houlsby, N., et al. (2019). *Parameter-Efficient Transfer Learning for NLP*. ICML 2019.
- [5] Ben-Zaken, E., et al. (2022). *BitFit : Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-Models*. ACL 2022.
- [6] Jeffrey O. Zhang, Alexander Sax, Amir Zamir, Leonidas J. Guibas, Jitendra Malik 2019. A Statistical Side-tuning : Network Adaptation via Additive Side Networks
- [7] B.S. Veeling et al. Rotation Equivariant CNNs for Digital Pathology. MICCAI 2018.

ANNEXES

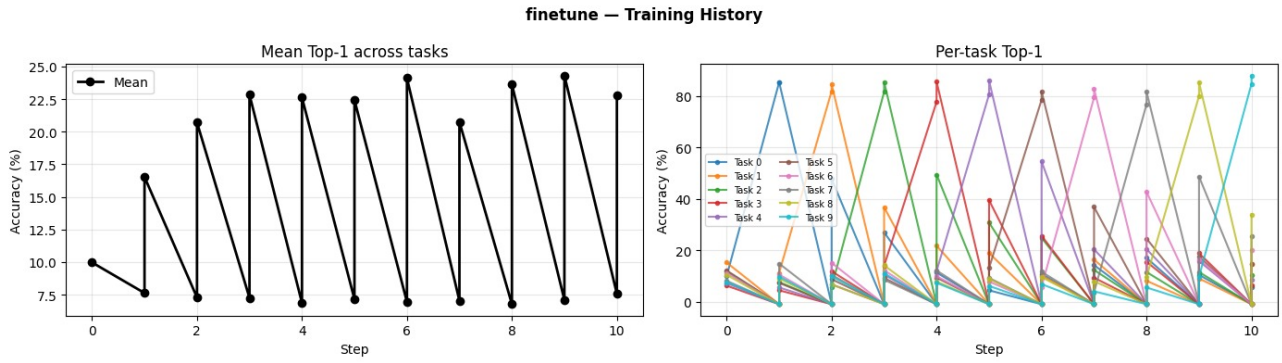


FIGURE 3 – Fine-tuning — Historique d'entraînement et accuracy par tâche.

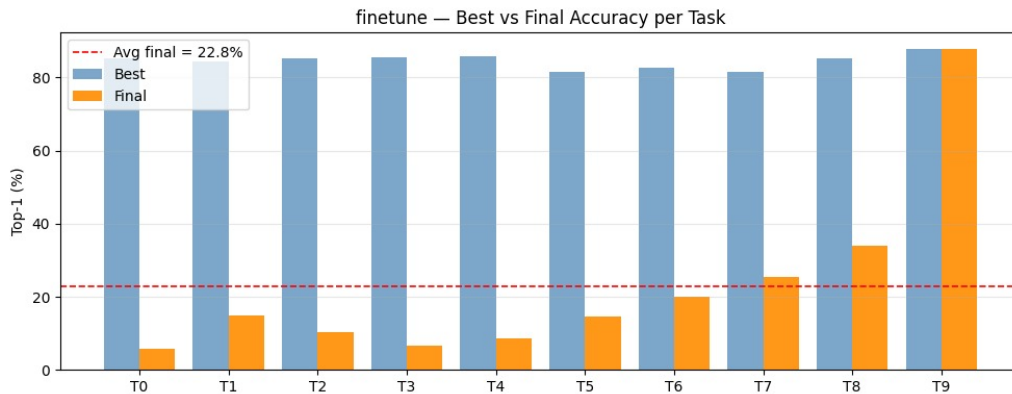


FIGURE 4 – Fine-tuning — Meilleure accuracy vs accuracy finale par tâche.

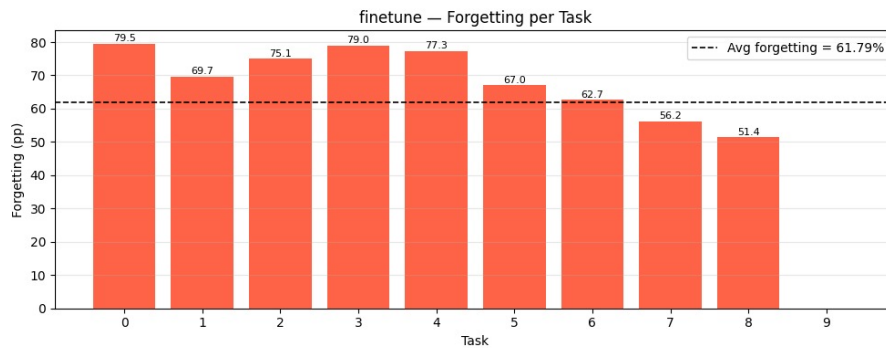


FIGURE 5 – Fine-tuning — Oubli par tâche (en points de pourcentage).

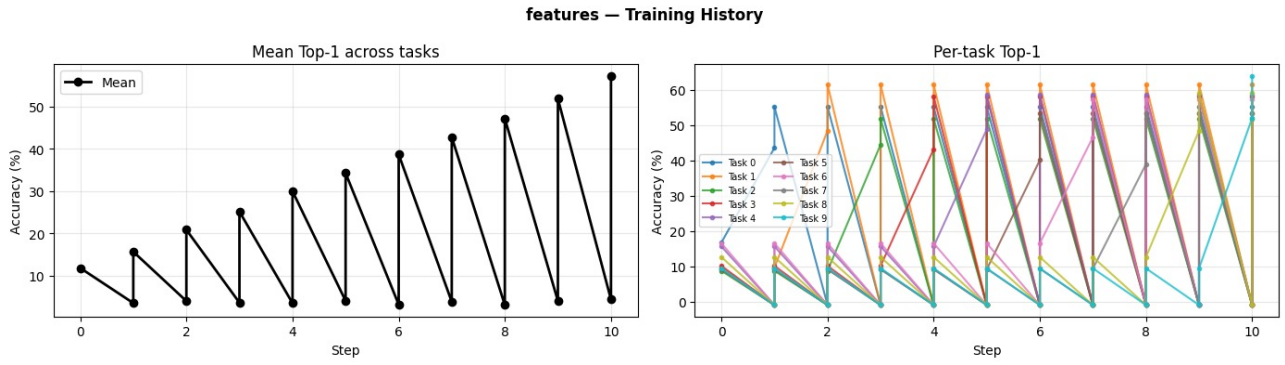


FIGURE 6 – Feature Extraction — Historique d’entraînement et accuracy par tâche.

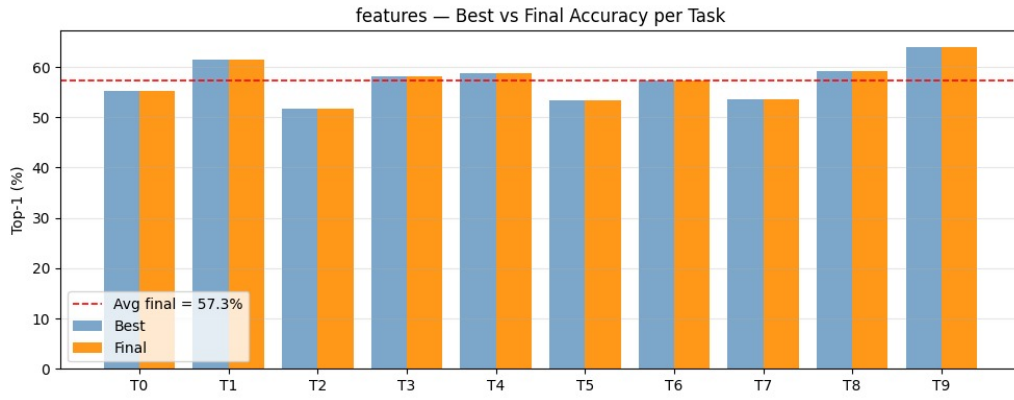


FIGURE 7 – Feature Extraction — Meilleure accuracy vs accuracy finale par tâche.

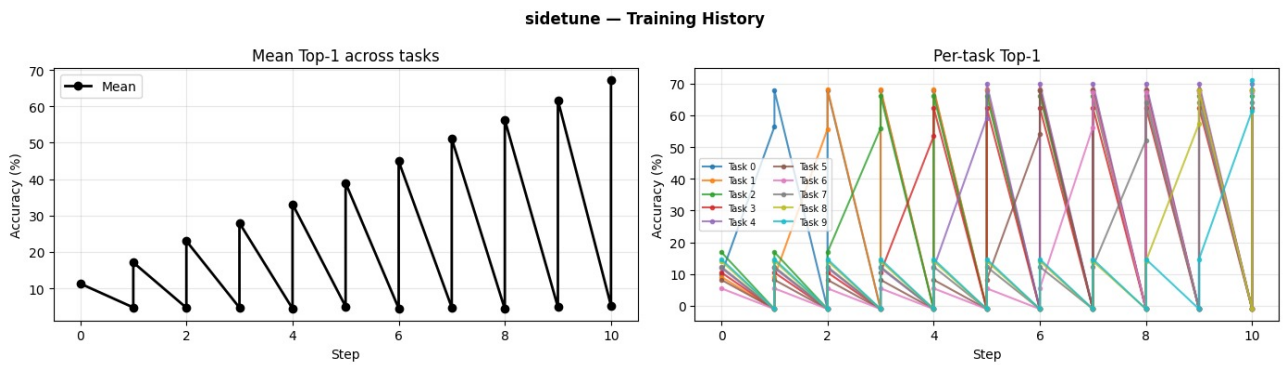


FIGURE 8 – Side-Tuning — Historique d’entraînement et accuracy par tâche.

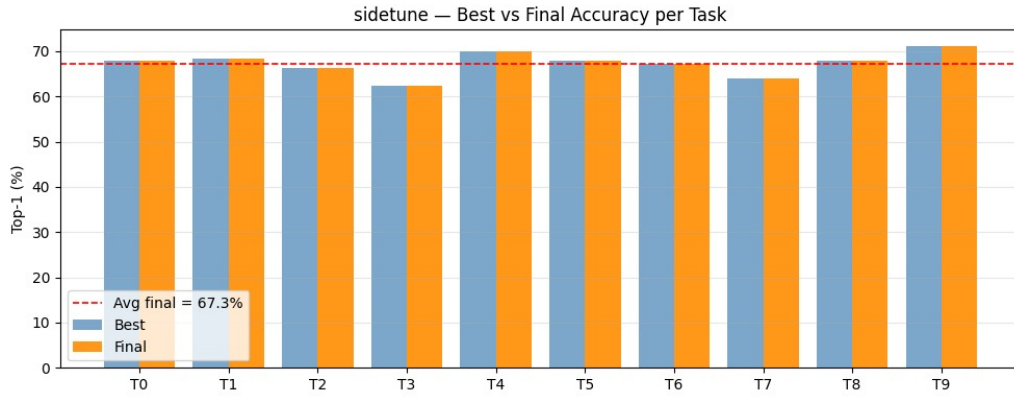


FIGURE 9 – Side-Tuning — Meilleure accuracy vs accuracy finale par tâche.

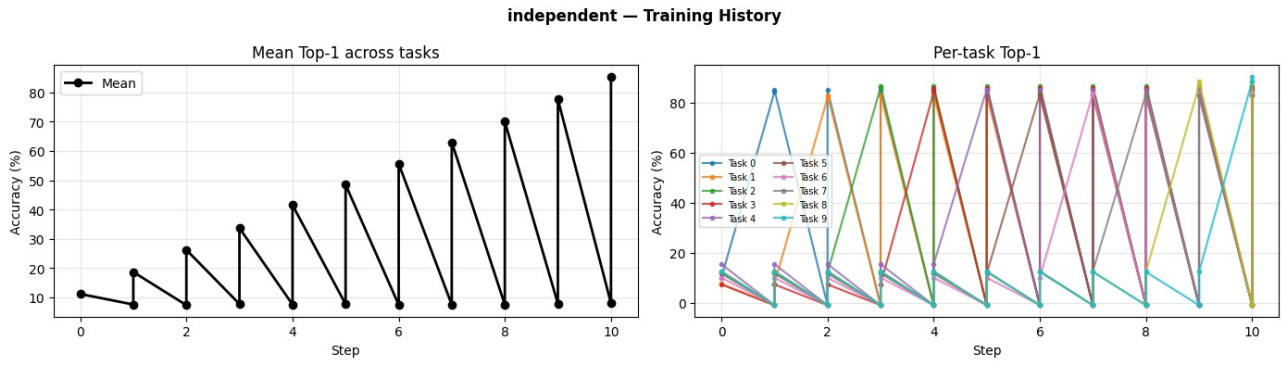


FIGURE 10 – Independent — Historique d'entraînement et accuracy par tâche.

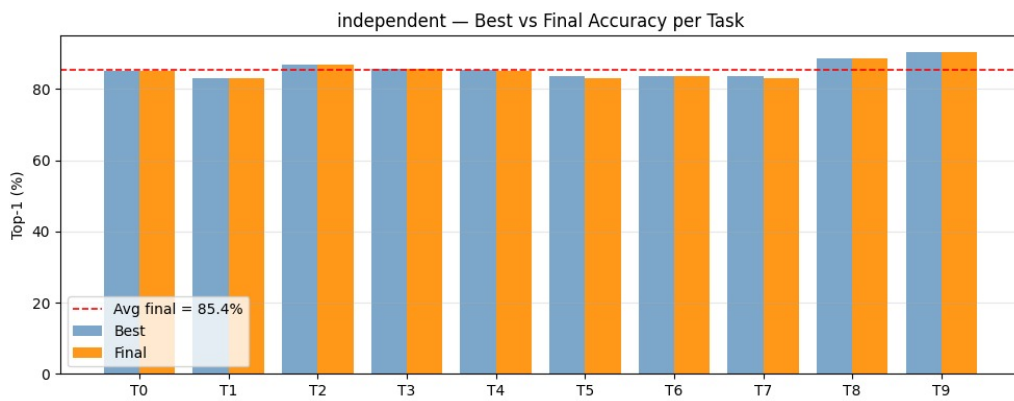


FIGURE 11 – Independent — Meilleure accuracy vs accuracy finale par tâche.

Exemples d'images PatchCamelyon (96×96 px)

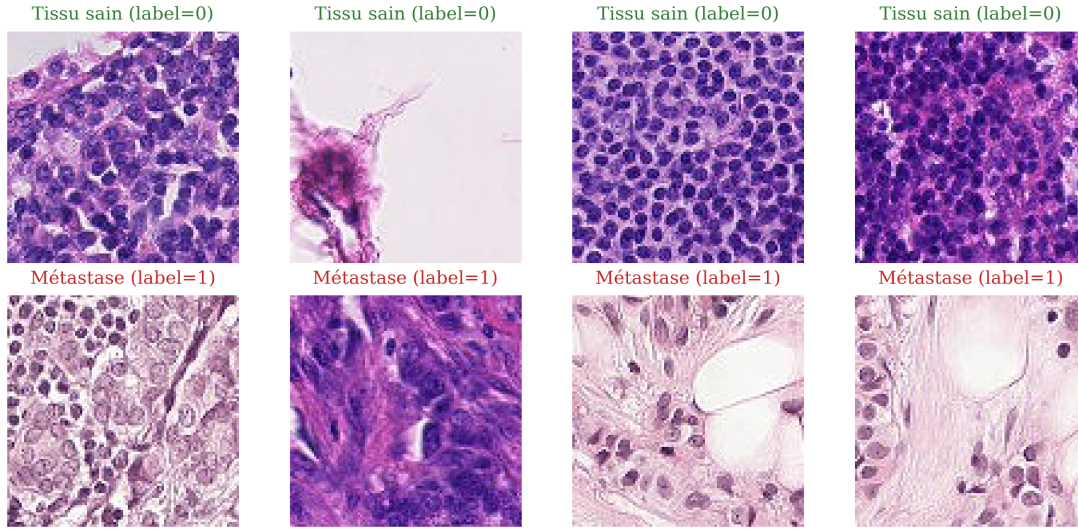


FIGURE 12 – Exemples d'images PatchCamelyon (96 × 96 px). **Haut** : tissu sain (label=0). **Bas** : tissu métastatique (label=1). Coloration hématoxyline-éosine (H&E).

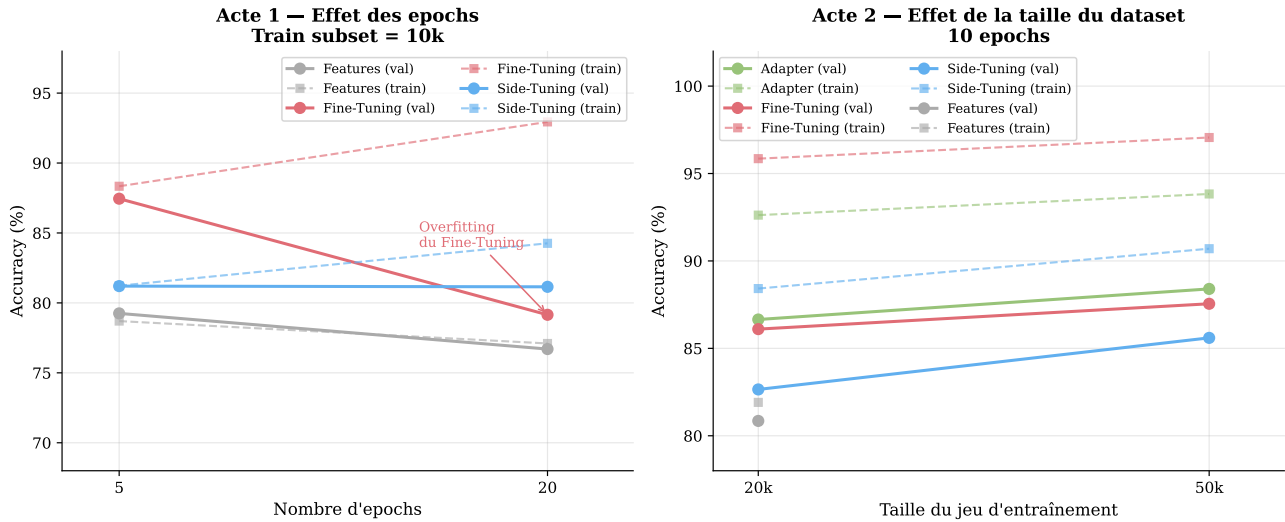


FIGURE 13 – **Gauche** : val et train accuracy en fonction des epochs (10k images). Le fine-tuning chute de 87.5% à 79.2% entre 5 et 20 epochs (overfitting). Side-tuning reste stable. **Droite** : val et train accuracy en fonction de la taille du dataset (10 epochs). Toutes les méthodes progressent avec plus de données.

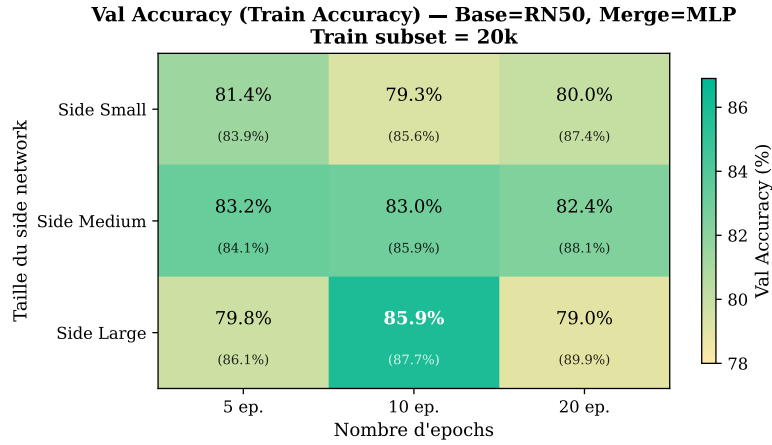


FIGURE 14 – Val accuracy (train accuracy entre parenthèses) pour la grille side size $\times$ epochs, avec Base=ResNet-50 et Merge=MLP (train subset = 20k). Meilleure config : Side Large / 10 epochs $\rightarrow$ 85.9% (87.7%).

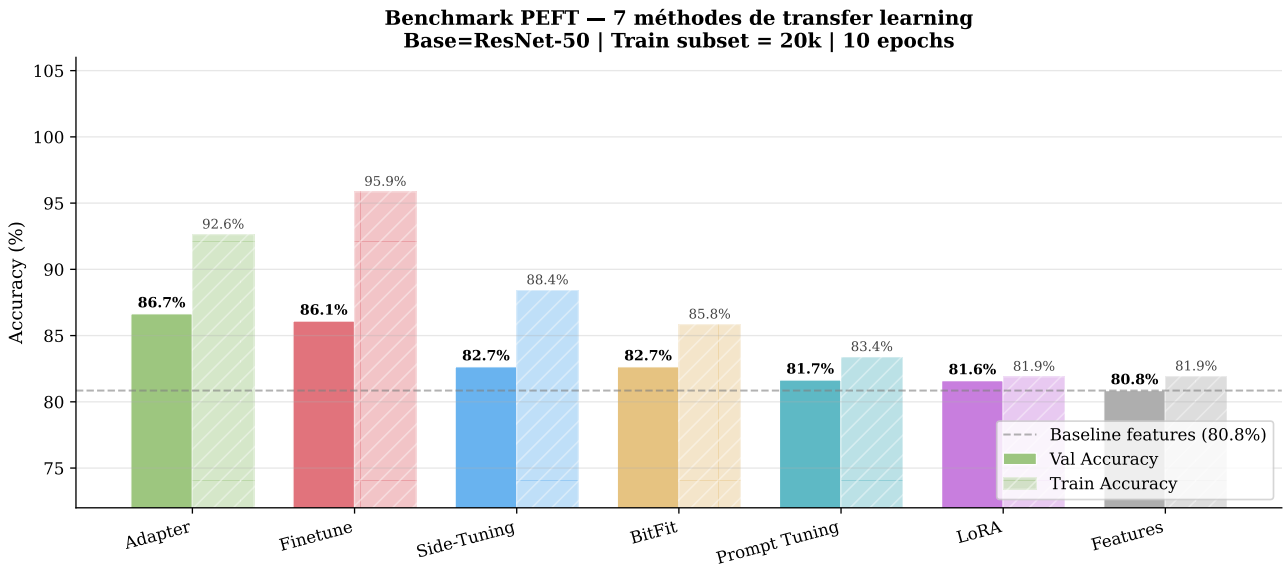


FIGURE 15 – Val et train accuracy des 7 méthodes PEFT (Base=ResNet-50, 20k train, 10 epochs).

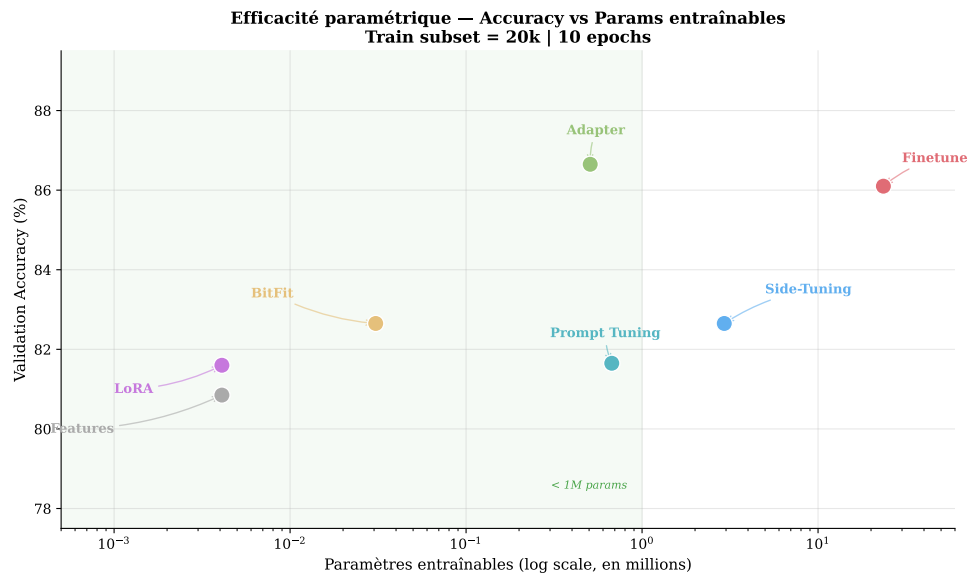


FIGURE 16 – Efficacité paramétrique : val accuracy en fonction du nombre de paramètres entraînaibles (échelle log). Adapter Layers offre le meilleur rapport accuracy / paramètres parmi les méthodes compétitives.